

Lab 1: Python Programming, 2D Drone PID Control and Waypoint Tracking

Abstract

This lab focuses on the Python programming basics with a simulated 2D drone within the so-called *RetroFlight* environment, to be prepared for the real-world hardware control (also using Python). The task combines low-level PID development (in a safe and controlled environment) with high-level offboard mission control using Python (managed via *venvs*). The primary goal is to implement a robust PID controller capable of compensating for environmental disturbances such as wind, and to perform waypoint planning.

Key Objectives

- **Position PID Control:** Implementation of a direct PID position controller. This single-tier system calculates the required thrust directly from the positional error to navigate the drone and actively reject wind disturbances.
- **Offboard Mission Control:** Commanding the simulated drone via a separate high-level script (`my_mission.py`) that interacts with the simulation over TCP/IP using the `retro_cflib` API.
- **Waypoint Sequencing:** Programming an autonomous flight sequence directing the drone to take off, precisely fly a $5\text{ m} \times 5\text{ m}$ square pattern, and return to its starting point.
- **Autonomous Navigation & Interaction:** Extracting environment maps and spatial coordinates to plan routes, enabling the drone to autonomously locate and collect batteries during the mission.

Software Setup

Getting the project files. The project is hosted on the h_da GitLab:

<https://code.fbi.h-da.de/jan.zwiener/retroflight>

Choose one of the following methods:

Option A – Git clone (recommended):

```
git clone https://code.fbi.h-da.de/jan.zwiener/retroflight.git
cd retroflight
```

Option B – Download as ZIP (no Git required):

<https://code.fbi.h-da.de/jan.zwiener/retroflight/-/archive/main/retroflight-main.zip>

Extract the ZIP and open a terminal in the extracted folder.

Setup and Execution.

Windows: (requires Python 3.8+ installed and added to PATH).

- double click on `setup.bat` (one time setup)
- to run the simulation: `run.bat`

macOS / Linux:

```
source setup.sh    # (once)
source env.sh
python -m retroflight.main
```

Low-Level Firmware (controller.py). The drone calls `compute_thrust()` at every simulation step.

```
def compute_thrust(self, state, setpoint, dt, time):
    """
    state:      np.array [x, y, z, vx, vy, vz, ax, ay, az] (m, m/s, m/s2)
    setpoint:   np.array [target_x, target_y, target_z]      (m)
    returns:    np.array [thrust_x, thrust_y, thrust_z]      (m/s2)
    """
```

Implement the PID control law inside `controller.py` to compute a thrust vector that drives the drone toward the `setpoint` position.

High-Level Offboard Control (my_mission.py). Open a **second terminal** and run your mission script from the `external_control/` directory:

```
cd external_control
python my_mission.py
```

The mission script communicates with the running simulation via TCP/IP on `localhost:5000` using the `retro_cflib` mock library, which mirrors the Crazyflie Python API.

Preparation

Only two files are relevant for this lab. Everything else is engine code in `src/retroflight/_engine/` and should not be modified. See figure 1 for a high-level block diagram.

File	Purpose
<code>src/retroflight/controller.py</code>	Low-level flight controller (firmware)
<code>external_control/my_mission.py</code>	High-level mission script (offboard)

- Read `controller.py` and understand the structure of the `state` and `setpoint` vectors.
- Derive the PID control law on paper: write down the expressions for proportional, integral, and derivative terms.
- Prepare a Python implementation of the position PID controller as a skeleton (no tuning required yet).
- Familiarize yourself with the available commands in the `retro_cflib.py` API for commanding the drone and retrieving state information in `my_mission.py`.

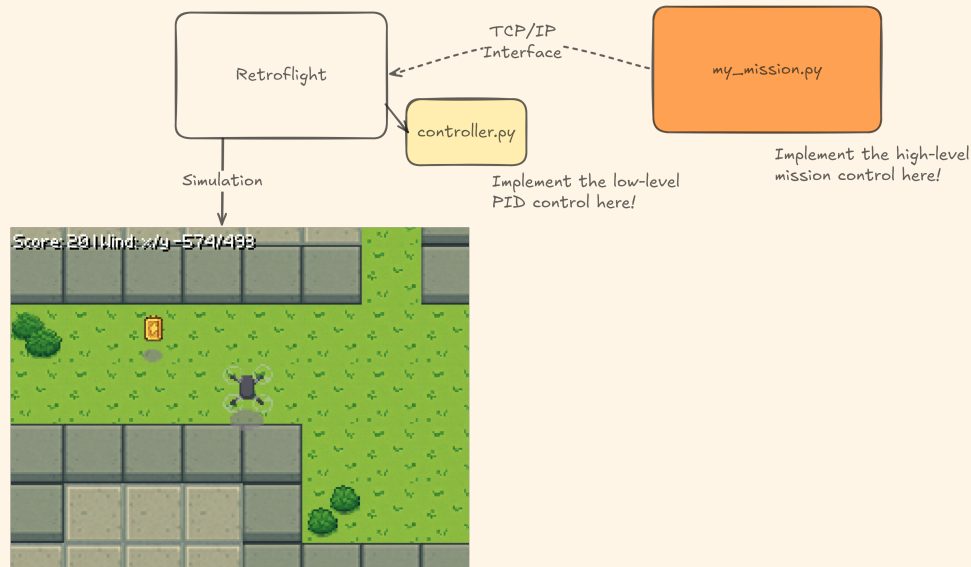


Figure 1: TCP/IP Interface between low-level controller and mission planner.

Lab Tasks

1. **Implement the PID Controller.** Open `controller.py` and implement the `compute_thrust()` method. Design a **single-loop position PID controller**: use the error between the **setpoint** and the current position to compute the proportional and integral terms, and use the current velocity for the derivative term to avoid derivative kick. Ensure the controller handles the **simulated wind**.

Advanced Extension (Optional). Instead of a single-loop design, you may implement a **cascaded control architecture**. This typically involves an outer loop that calculates a target velocity based on the position error, and an inner loop that calculates the final thrust based on the velocity error. This approach generally yields smoother and more stable flight dynamics.

2. **Waypoint Sequencer.** Edit `my_mission.py` to command the drone to take off, fly a $5\text{m} \times 5\text{m}$ square pattern, and return to the start position. Use `cf.commander.go_to(x, y, z)` for each corner. The `go_to()` call blocks until the drone has arrived within a configurable tolerance.

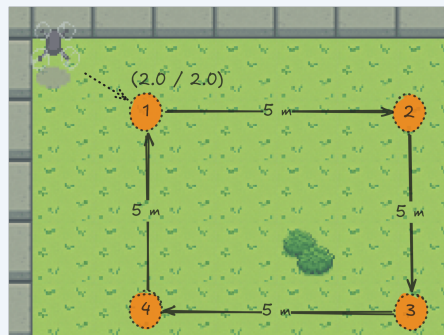


Figure 2: Square waypoint pattern flown by the drone.

3. **Autonomous Battery Collection.** Use `cf.commander.get_batteries()`



to retrieve the positions of all batteries on the map. Use `cf.commander.get_map()` to retrieve the obstacle map. Extend your mission script to fly to each battery and collect it. Use `cf.commander.get_state()` to monitor the current position and score during the mission.

Deliverables

- **Demonstration:** The drone completes the square waypoint pattern with sub-meter accuracy and collects at least three batteries autonomously.

```
# --- retroFlight/controller.py ---
import numpy as np

class UAVController:
    def __init__(self):
        self.Kp = np.array([ 0.0, 0.0, 0.0])
        self.Ki = np.array([ 0.0, 0.0, 0.0])
        self.Kd = np.array([ 0.0, 0.0, 0.0])

        self.error_sum = np.zeros(3)
        self.Last_error = np.zeros(3)
        self.thrust = np.zeros(3)

    def compute_thrust(self, state, setpoint, dt, time):
        """
        Calculate thrust for vehicle.
        x, y, z in meter
        vx, vy, vz in m/s
        and ax, ay, az m/s^2

        :param state: np.array [x, y, z, vx, vy, vz, ax, ay, az]
        :param setpoint: np.array [target_x, target_y, target_z]
        :param dt: delta time
        :param time: absolute simulation time
        :return: np.array [thrust_x, thrust_y, thrust_z]
        """
        pos = state[0:3]
        vel = state[3:6]

        error = setpoint - pos
        error[2] = 0.0 # ignore altitude for now

        # --- PID ---
        #
        #
        return self.thrust
```

Add implementation here!

Figure 3: Source code section for the PID controller implementation.

```
# --- my_mission.py ---
import time

from retro_cflib import CrazyflieMock as Crazyflie

def main():
    print("Starting mission...")
    cf = Crazyflie()

    # Get the map
    world = cf.commander.get_map()
    grid = world["grid"]
    # Test if a tile is blocked or free:
    # if grid[1][1] == "##":
    #     print("blocked")
    # else:
    #     print("free")
    # for row in grid:
    #     print(row)

    # Get batteries:
    batteries = cf.commander.get_batteries()
    for b in batteries:
        print(f"Battery at ({b['x']:.1f}, {b['y']:.1f})")

    cf.commander.go_to(x=1.0, y=1.0, z=0.1)
    cf.commander.go_to(x=3.0, y=3.0, z=1.0)
    print("Mission end.")

if __name__ == '__main__':
    main()
```

Add implementation here!

Figure 4: Source code section for the high-level motion planning implementation.